

Reconocimiento de tramas I²C con analizador lógico

Riveros Sierra Harol Felipe 1401660, Bohorquez Blanco Salome 1401654

Docente: Ing. José de Jesús Rugeles Uribe.

I. RESUMEN

En esta práctica de laboratorio se analizó el protocolo de comunicación I²C mediante una Raspberry Pi Pico 2W, una pantalla OLED SSD1306 y un analizador lógico utilizando el software Logic 2. Se identificaron los elementos que conforman una trama I²C, incluyendo las condiciones de inicio y parada, el direccionamiento de 7 bits y los estados de reconocimiento ACK y NACK. Asimismo, se realizaron pruebas de comunicación con direcciones correctas e incorrectas, un escaneo de dispositivos conectados al bus y la observación de comandos enviados a la pantalla OLED. Finalmente, se estableció la relación entre las señales digitales adquiridas y la información decodificada, permitiendo reconocer la estructura y el funcionamiento de las transacciones del protocolo.

II. ABSTRACT

In this laboratory exercise, the I²C communication protocol was analyzed using a Raspberry Pi Pico 2W, an SSD1306 OLED display, and a logic analyzer with Logic 2 software. The components of an I²C frame were identified, including start and stop conditions, 7-bit addressing, and ACK/NACK acknowledgment states. Additionally, communication tests were conducted using both correct and incorrect addresses; a scan of devices connected to the bus was performed; and commands sent to the OLED display were observed. Finally, the relationship between the captured digital signals and the decoded information was established, enabling an understanding of the structure and operation of the protocol's transactions.

III. PALABRAS CLAVE

I²C, Raspberry Pi Pico 2W, Logic 2, ACK, NACK, Bits de parada, Bit de inicio, Pantalla OLED

IV. KEYWORDS

I²C, Raspberry Pi Pico 2W, Logic 2, ACK, NACK, Stop Bits, Start Bit, OLED Display

V. INTRODUCCIÓN

Los protocolos de comunicación serial permiten el intercambio de información entre microcontroladores y dispositivos periféricos mediante señales digitales. Entre estos protocolos se encuentra I²C, el cual utiliza las líneas SDA y SCL para transmitir datos y sincronizar las operaciones entre los dispositivos conectados al bus.

El reconocimiento de las tramas de comunicación es fundamental para verificar el direccionamiento de los dispositivos, la transmisión de información y las respuestas generadas durante una transacción. En este contexto, el analizador lógico constituye una herramienta que permite visualizar las señales digitales y decodificar los datos transmitidos, facilitando la identificación de las condiciones START y STOP, los bits de dirección y los estados ACK/NACK.

En el presente laboratorio se empleó una Raspberry Pi Pico 2W programada en MicroPython para establecer comunicación con una pantalla OLED SSD1306 mediante el protocolo I²C. A través del software Logic 2 se capturaron y analizaron las señales del bus, realizando pruebas de reconocimiento de direcciones, verificación de respuestas y observación de comandos enviados a la pantalla.

De esta manera, se estudió experimentalmente la estructura de las transacciones I²C y se relacionaron las señales adquiridas con las operaciones ejecutadas por el microcontrolador sobre el dispositivo periférico.

VI. OBJETIVOS

Identificar las condiciones START y STOP, el direccionamiento de 7 bits y los estados ACK/NACK en las tramas I²C mediante el software Logic 2.

Verificar la respuesta de la pantalla OLED SSD1306 utilizando direcciones correctas e incorrectas y comparar las capturas obtenidas.

Determinar la frecuencia de reloj SCL mediante la medición del período de la señal adquirida con el analizador lógico.

Implementar el escaneo de direcciones I²C mediante MicroPython para detectar los dispositivos conectados al bus.

Reconocer los comandos y datos transmitidos a la pantalla OLED, relacionando los códigos hexadecimales observados con sus respectivas funciones.

VII. MARCO TEÓRICO

A. PROTOCOLO DE COMUNICACIÓN I²C

El protocolo Inter-Integrated Circuit (I²C) es un protocolo de comunicación serial síncrona utilizado para intercambiar información entre un microcontrolador y diferentes dispositivos periféricos. Su funcionamiento se basa principalmente en dos líneas de comunicación: SDA (Serial Data), por donde se transmiten los datos, y SCL (Serial Clock), que proporciona la señal de reloj utilizada para sincronizar la comunicación.

En una comunicación I²C, el dispositivo que inicia y controla la transferencia se denomina maestro, mientras que los dispositivos que reciben las solicitudes se denominan esclavos. Cada dispositivo conectado al bus posee una dirección que permite al maestro identificar con cuál de ellos desea comunicarse.

En esta práctica, la Raspberry Pi Pico 2W actúa como maestro y se comunica con una pantalla OLED basada en el controlador SSD1306. Se utiliza la interfaz I²C1, configurando el pin GP15 como SCL y el pin GP14 como SDA, con una frecuencia de reloj de 100 kHz. Estas señales son adquiridas mediante un analizador lógico y posteriormente visualizadas y decodificadas en el software Logic 2.

El propósito del análisis es observar las señales eléctricas del bus y relacionarlas con los datos que realmente se están transmitiendo. De esta manera, es posible comprobar si el dispositivo recibe correctamente la información y si responde a las solicitudes realizadas por el microcontrolador.

B. ESTRUCTURA DE UNA TRAMA I²C

Una transacción I²C está formada por una secuencia de señales que permite iniciar la comunicación, identificar el dispositivo con el que se desea establecer comunicación, indicar el tipo de operación y verificar si el dispositivo receptor respondió correctamente.

Antes de iniciar una comunicación, el bus se encuentra normalmente en estado IDLE, en el cual las líneas SDA y SCL permanecen en nivel alto. A partir de este estado se pueden identificar los siguientes elementos:

- CONDICION DE START

La condición START indica el comienzo de una transacción I²C. Se produce cuando la línea SDA cambia de nivel alto a nivel bajo mientras SCL permanece en nivel alto.

Esta transición es importante porque permite que los dispositivos conectados al bus reconozcan que una nueva comunicación está comenzando.

- DIRECCION Y BIT R/W

Después de la condición START, el maestro transmite la dirección del dispositivo con el que desea comunicarse. En este laboratorio se utiliza un direccionamiento de 7 bits.

La pantalla OLED utilizada tiene la dirección:

$$\text{ADDR} = 0x3C$$

En binario, esta dirección corresponde a:

$$0x3C = 0111100_2$$

A los 7 bits de dirección se agrega un octavo bit denominado R/W, que indica el tipo de operación:

$$\begin{aligned} \text{R/W} = 0: & \text{ escritura.} \\ \text{R/W} = 1: & \text{ lectura.} \end{aligned}$$

Para una operación de escritura, la dirección de 7 bits se desplaza una posición hacia la izquierda y se agrega un cero como bit R/W:

$$\text{Octeto} = (0x3C \ll 1) | 0$$

$$\text{Octeto} = 0x78$$

Por esta razón, aunque la dirección del dispositivo es 0x3C, en el analizador lógico puede observarse el octeto 0x78 cuando se realiza una operación de escritura. La guía del laboratorio establece precisamente la identificación de la secuencia: START → 0x78 → ACK → STOP.

C. BIT DE RECONOCIMIENTO ACK/NACK

Después de transmitir los ocho bits correspondientes a la dirección y al bit R/W, se genera un noveno pulso de reloj utilizado para indicar si el dispositivo receptor reconoció la comunicación.

Durante este noveno pulso, el maestro libera la línea SDA para permitir que el dispositivo receptor genere la respuesta.

Si el dispositivo reconoce la dirección recibida, coloca SDA en nivel bajo, generando un ACK.

Por el contrario, si ningún dispositivo reconoce la dirección, SDA permanece en nivel alto, generando un NACK.

Esta diferencia permite comprobar experimentalmente si una dirección corresponde o no a un dispositivo conectado al bus.

D. CONDICION DE STOP

La condición STOP indica el final de una transacción I²C. Se produce cuando la línea SDA cambia de nivel bajo a nivel alto mientras SCL permanece en nivel alto.

Por lo tanto, una comunicación básica de escritura puede representarse de forma simplificada como:

START → Dirección + R/W → ACK/NACK → STOP

Esta estructura es la que se busca identificar visualmente mediante el analizador lógico.

E. DIRECCIONAMIENTO Y RECONOCIMIENTO ACK/NACK

El direccionamiento es el mecanismo mediante el cual el maestro selecciona el dispositivo con el que desea establecer comunicación. En el presente laboratorio, la pantalla OLED SSD1306 utiliza la dirección 0x3C.

Para comprobar el funcionamiento del direccionamiento se realizan dos pruebas. En la primera se utiliza la dirección correcta 0x3C, por lo que se espera obtener una respuesta ACK. En la segunda se utiliza una dirección que no corresponde a un dispositivo presente en el bus, como 0x3D, con el objetivo de generar una respuesta NACK.

Cuando se utiliza la dirección correcta, la secuencia observada en Logic 2 debe corresponder aproximadamente a:

START → 0x78 → ACK → STOP

En cambio, al utilizar una dirección incorrecta, se mantiene la estructura de la comunicación, pero durante el noveno pulso de reloj, SDA permanece en nivel alto, indicando que ningún dispositivo reconoció la dirección.

La comparación entre ambas capturas permite comprobar que la principal diferencia se encuentra en el noveno pulso de reloj, donde se presenta ACK para una dirección válida y NACK para una dirección que no corresponde a un dispositivo conectado.

F. ANALIZADOR LÓGICO Y MEDICION DE LA FRECUENCIA SCL

Un analizador lógico es un instrumento utilizado para adquirir y visualizar señales digitales en función del tiempo. En esta práctica permite observar las líneas SCL y SDA y utilizar el software Logic 2 para interpretar la información transmitida mediante el protocolo I²C.



Figura 1. Analizador lógico empleado en la práctica de laboratorio.

Para realizar la captura, el analizador lógico debe conectarse a las líneas correspondientes del bus y compartir la misma referencia de tierra con la Raspberry Pi Pico 2W.

De acuerdo con la guía, la señal SCL corresponde al pin GP15 y la señal SDA al pin GP14.

Una vez realizada la conexión, se habilita el decodificador I²C en Logic 2. Este permite representar la información de la señal de manera más sencilla, mostrando eventos como Start, Write, dirección, ACK/NACK y Stop, además de los datos transmitidos.

• FRECUENCIA DE MUESTREO

Para que el analizador lógico pueda representar adecuadamente una señal digital, su frecuencia de muestreo debe ser superior a la frecuencia de la señal que se desea analizar. La guía establece como referencia una frecuencia de muestreo de al menos 10 veces a frecuencia del bus I²C.

Para este laboratorio:

$$f_{SCL} = 100 \text{ kHz}$$

Por lo tanto:

$$f_{\text{muestreo}} \geq 10 \times f_{SCL}$$

$$f_{\text{muestreo}} \geq 10 \times 100 \text{ kHz}$$

$$f_{\text{muestreo}} \geq 1 \text{ MHz}$$

En consecuencia, para una frecuencia de reloj I²C de 100 kHz, el analizador debe configurarse con una frecuencia de muestreo de 1 MS/s o superior.

- MEDICION DE LA FRECUENCIA DE SCL

Además de observar la información decodificada, el analizador lógico permite medir directamente el período de la señal SCL mediante los cursores de tiempo de Logic 2.

El período (T) corresponde al intervalo de tiempo entre dos puntos equivalentes consecutivos de la señal, por ejemplo, entre dos flancos de subida consecutivos.

Una vez obtenido el período, la frecuencia se calcula mediante:

$$f_{SCL} = \frac{1}{T_{SCL}}$$

Para una señal configurada a 100 kHz, el período teórico es:

$$T_{SCL} = \frac{1}{f_{SCL}} = \frac{1}{100\text{KHz}} = 10\mu\text{s}$$

Por lo tanto, al realizar la medición experimental, el período observado debe encontrarse aproximadamente alrededor de 10 μs , teniendo en cuenta las posibles variaciones propias de la adquisición y del sistema utilizado.

Esta medición permite comparar el valor configurado en el programa de la Raspberry Pi Pico 2W con el comportamiento real de la señal SCL observada en el analizador lógico.

G. RASPBERRY PI PICO 2W

La Raspberry Pi Pico 2W es una tarjeta de desarrollo de bajo costo basada en el microcontrolador RP2350, de arquitectura ARM Cortex-M33 de doble núcleo, que incorpora además un módulo de conectividad inalámbrica Wi-Fi/Bluetooth. La tarjeta expone en sus pines de propósito general (GPIO) varios periféricos de comunicación, entre ellos hasta dos módulos UART, dos buses I2C y dos buses SPI, los cuales pueden asignarse a distintos pines físicos según la necesidad del proyecto. Para esta práctica de laboratorio la Raspberry Pi Pico 2W es utilizada como sistema de adquisición, aprovechando su convertidor analógico-digital integrado para realizar mediciones de señales eléctricas.

En la siguiente figura se muestra la ubicación de los respectivos pines en La Raspberry Pi Pico 2W:

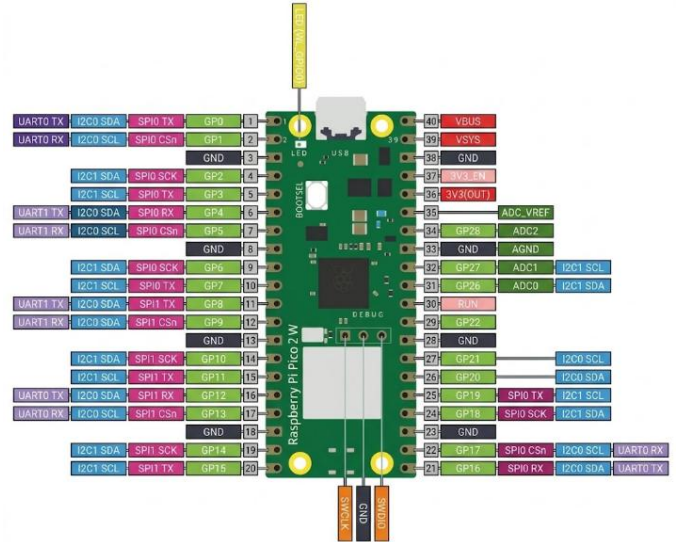


Figura 2. Ubicación de los pines en La Raspberry Pi Pico 2W.

VIII. METODOLOGÍA/PROCEDIMIENTO EXPERIMENTAL

A. MATERIALES Y MONTAJE

Se empleó una Raspberry Pi Pico 2W programada en MicroPython como maestro del bus I²C, una pantalla OLED SSD1306 de 128×32 píxeles con dirección 0x3C como dispositivo esclavo y un analizador lógico USB de 8 canales y 24 MHz controlado con el software Logic 2. Se utilizó la interfaz PC1 de la Pico, con SCL en GP15 y SDA en GP14 [6]. El canal CH0 del analizador se conectó a SCL y el canal CH1 a SDA, uniendo la tierra del analizador con la de la tarjeta. En Logic 2 se habilitó el decodificador I²C y la frecuencia de muestreo se configuró siguiendo el criterio de la guía, es decir, al menos 10 veces la frecuencia del bus ($\geq 1 \text{ MS/s}$ para 100 kHz).

B. PRUEBA DE RECONOCIMIENTO ACK/NACK

Se cargó en la Pico el script `OLED_ADDR_test.py`, que inicializa PC1 a 100 kHz, espera un segundo para armar la captura y ejecuta `i2c.writeto(ADDR, b"", True)` con `ADDR = 0x3C`. Con esta dirección se espera la secuencia `START → 0x78 → ACK → STOP`. En la captura se identificaron el `START`, los ocho bits del octeto, el noveno bit y el `STOP`, y se midió `fSCL` con los cursores de Logic 2. Para provocar un `NACK` se modificó `ADDR` a `0x3D`, una dirección sin dispositivo en el bus, y se repitió el procedimiento de captura, verificando el nivel de SDA durante el noveno pulso de reloj.

C. ESCANEADO DE DIRECCIONES

Se ejecutó `scan_i2c_addr.py`, que invoca `i2c.scan()` sobre PC1. Este método recorre las direcciones de 7 bits (`0x08–0x77`), envía `START + (ADDR << 1 | W)` para cada una, espera el `ACK` y devuelve la lista de las que respondieron [4]. La lista impresa

en consola se contrastó con la traza de Logic 2, identificando los sondeos con NACK y el sondeo con ACK.

D. COMANDOS ENVIADOS A LA PANTALLA OLED

Se ejecutó el programa OLED_demo_menu.py, que realiza un `i2c.scan()`, detecta la OLED en `0x3C` y despliega un menú por consola. En esta parte el bus se operó a 50 kHz. Para cada opción (apagar, encender, contraste, invertir y texto/animación) se capturó la transacción y se identificaron el byte de control, el comando o los datos y los ACK, comparándolos con la hoja de datos del SSD1306 [2]. Adicionalmente se fotografió la pantalla para verificar el efecto de cada comando.

IX. RESULTADOS

A. PRUEBAS ACK/NACK

Con `ADDR = 0x3C` Logic 2 decodificó la transacción como «Setup Write to [0x3C] + ACK». La Figura 3 muestra la condición START, en la que SDA pasa de nivel alto a bajo mientras SCL permanece en alto, y la Figura 4 el octeto `0x78`, formado por la dirección `0x3C` y el bit `R/W = 0`.

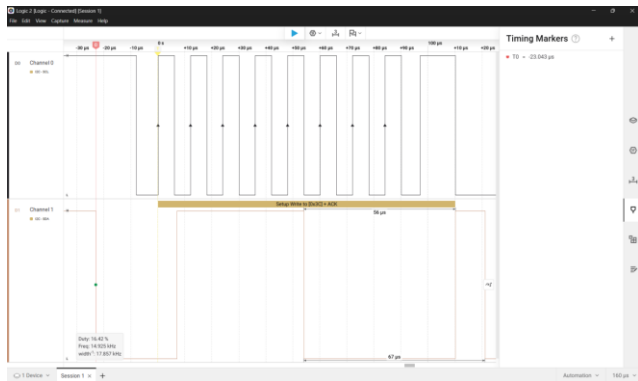


Figura 3. Condición START: SDA pasa de alto a bajo con SCL en alto (dirección 0x3C).

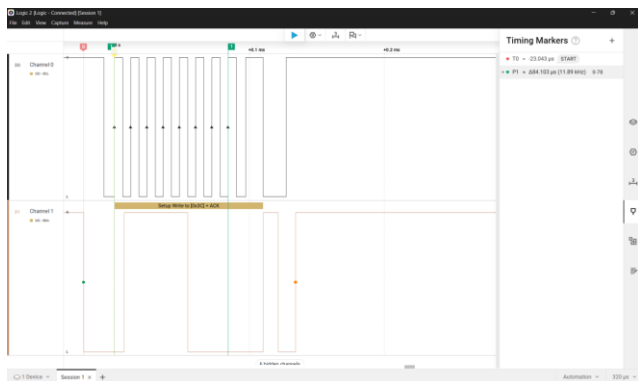


Figura 4. Octeto `0x78` (`0x3C << 1 | 0`) decodificado como Write to [0x3C].

En el noveno pulso la OLED fuerza SDA a nivel bajo (ACK, Figura 5) y la transacción termina con la condición STOP, donde SDA pasa de bajo a alto con SCL en alto (Figura 6).

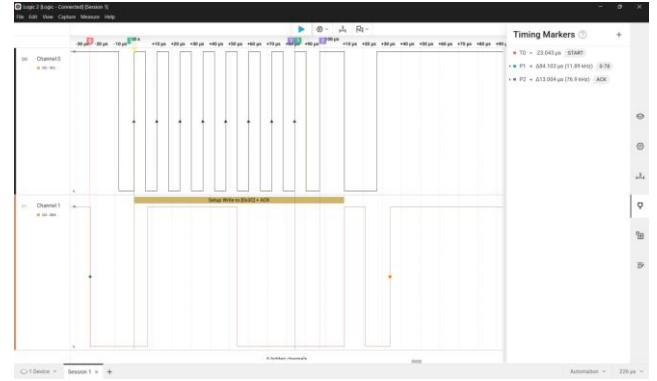


Figura 5. Noveno pulso de reloj: ACK generado por la OLED (SDA en bajo).

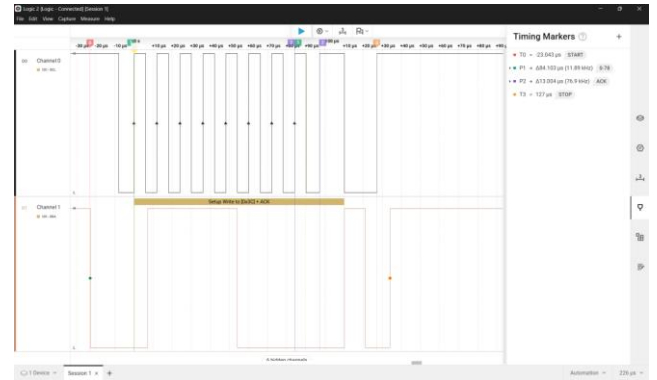


Figura 6. Condición STOP: SDA pasa de bajo a alto con SCL en alto.

La frecuencia de SCL se midió con los cursores de Logic 2 sobre un período de la señal (Figura 7), obteniéndose $12.038 \mu\text{s}$, equivalente a 83.07 kHz .

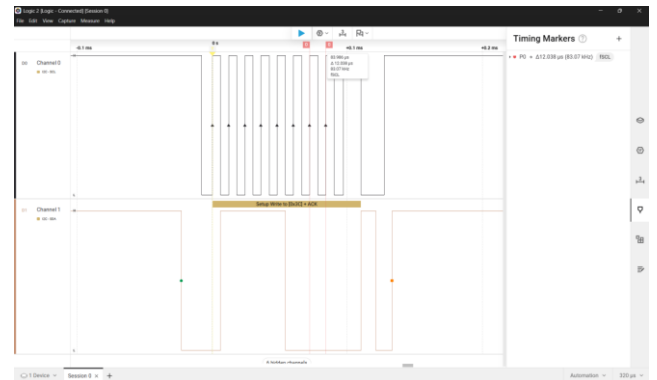


Figura 7. Medición de f_{SCL} con cursores: $12.038 \mu\text{s} = 83.07 \text{ kHz}$.

Al cambiar la dirección a `0x3D` el decodificador mostró «Setup Write to [0x3D] + NAK» (Figura 8). El octeto transmitido fue `0x7A` (`0x3D << 1 | 0`) y durante el noveno pulso SDA permaneció en nivel alto. La Figura 9 reúne los marcadores de tiempo de esta captura para compararla con la captura ACK.

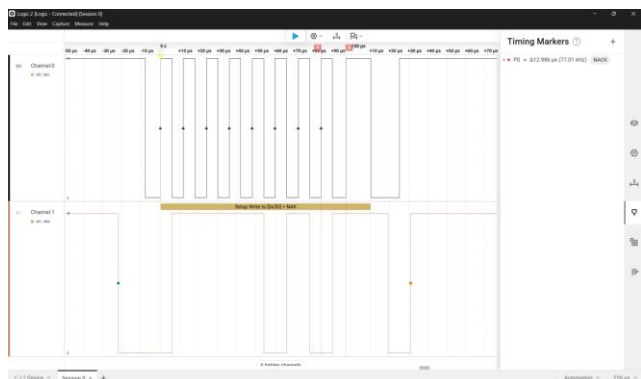


Figura 8. Captura con dirección incorrecta 0x3D: NACK en el bit 9.

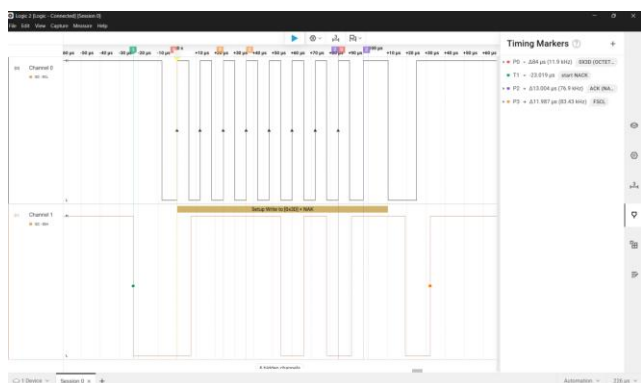


Figura 9. Captura NACK (0x3D) con marcadores de octeto, bit ACK/NACK y fSCL para la comparación con la captura ACK.

Tabla 1. Comparación entre las capturas ACK (0x3C) y NACK (0x3D).

Elemento	Captura ACK	Captura NACK
Start detectado (SDA↓ con SCL alto)	Sí	Sí
Octeto enviado / etiqueta del decodificador	0x78 / Write to [0x3C] + ACK	0x7A / Write to [0x3D] + NAK
Bit 9 (ACK = 0 / NACK = 1)	ACK (SDA = 0)	NACK (SDA = 1)
Stop detectado (SDA↑ con SCL alto)	Sí	Sí
Frecuencia medida de SCL	83.07 kHz	83.43 kHz

Tabla 2. Mediciones de tiempo realizadas con los cursores de Logic 2.

Medición con cursores	Tiempo	Frecuencia equivalente
Octeto 0x78 (8 bits)	84.103 μ s	95 kHz (10.5 μ s/bit)

Bit 9, captura ACK	13.004 μ s	76.9 kHz
Bit 9, captura NACK	12.986 μ s	77.01 kHz
Período de SCL, captura ACK	12.038 μ s	83.07 kHz
Período de SCL, captura NACK	11.987 μ s	83.43 kHz
Período de SCL en el menú OLED (configurado a 50 kHz)	21 μ s	47.6 kHz

B. ESCANEADO DE DIRECCIONES

Durante la ejecución de `i2c.scan()` el microcontrolador envió solicitudes de escritura a direcciones consecutivas. En la Figura 10 se observan los sondeos a 0x3B y 0x3D, con «NAK» en el noveno bit, y el sondeo a 0x3C, con «ACK». Entre el STOP de un sondeo y el START del siguiente transcurren aproximadamente 17–19 μ s. La consola de Thonny confirmó el resultado:

```
>>> %Run -c $EDITOR_CONTENT
I2C scan: ['0x3c']
```

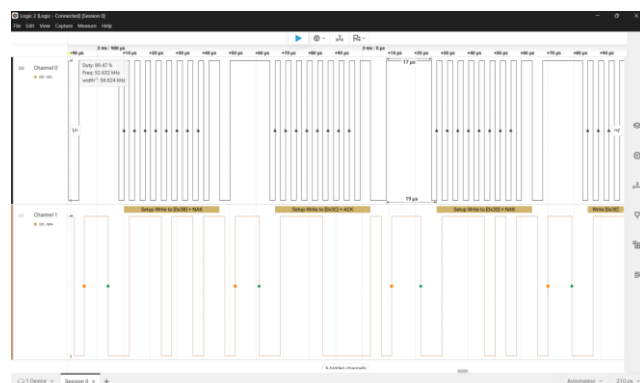


Figura 10. Traza de `i2c.scan()`: NAK en 0x3B y 0x3D, ACK en 0x3C.

C. COMANDOS DE LA PANTALLA OLED

Al ejecutar el programa del menú se detectó la OLED en 0x3C y se mostraron las opciones de la consola:

I2C scan: ['0x3c']

Usando OLED en addr: 0x3c

=== MENU OLED I2C (freq=50000 Hz, 128x32) ===

1. Apagar (0xAE)
2. Encender (0xAF)
3. Contraste (0-255)
4. Invertir 1/0
5. Limpiar
6. Texto demo

7. Animación breve
8. Enviar COMANDO RAW
9. Enviar DATO RAW
- F. Cambiar frecuencia I2C
0. Salir

Apagar (opción 1). La transacción observada fue START, Write [0x3C], byte de control 0x80 y comando 0xAE, todos con ACK (Figura 11). La pantalla quedó en negro (Figura 12).

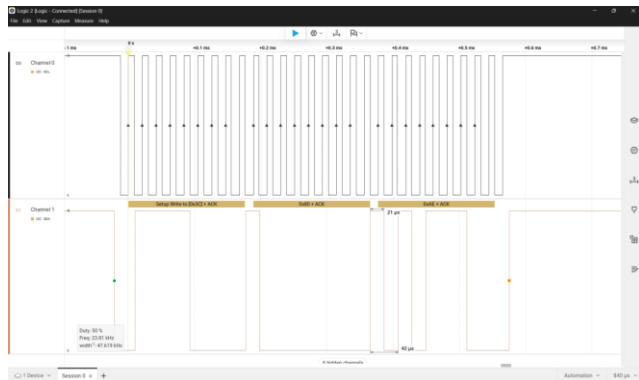


Figura 11. Opción 1, apagar: 0x80 + 0xAE (Set Display OFF).

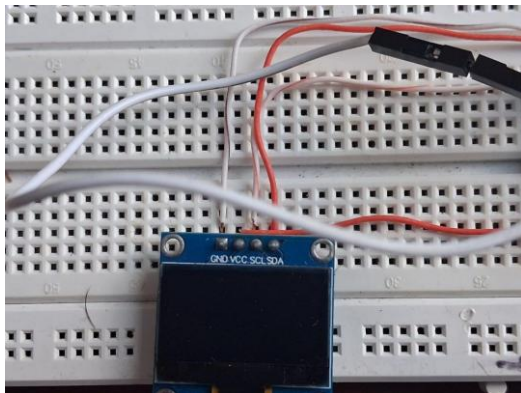


Figura 12. Pantalla OLED apagada.

Encender (opción 2). Se transmitió 0x80 seguido de 0xAF (Figura 13) y la pantalla mostró de nuevo el texto «I2C MENU / Addr 0x3c / Hola UMNG!» (Figura 14).

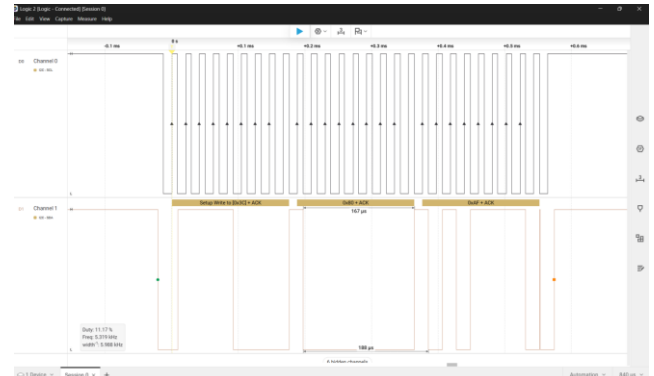


Figura 13. Opción 2, encender: 0x80 + 0xAF (Set Display ON).

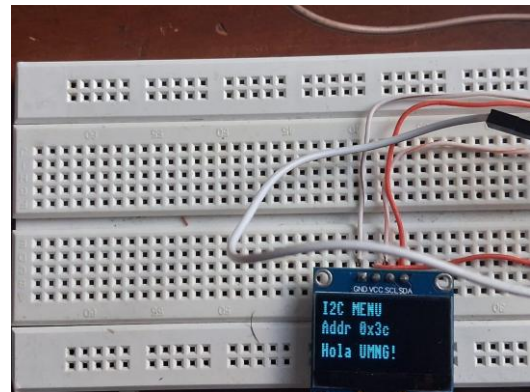


Figura 14. Pantalla OLED encendida.

Contraste (opción 3, valor 128). El comando se envió en dos transacciones separadas unos 2.15 ms (Figura 15). En la primera se transmitió 0x80+0x81 (Set Contrast Control, Figura 16) y en la segunda 0x80+0x80, donde 0x80 es el valor 128 elegido (Figura 17).

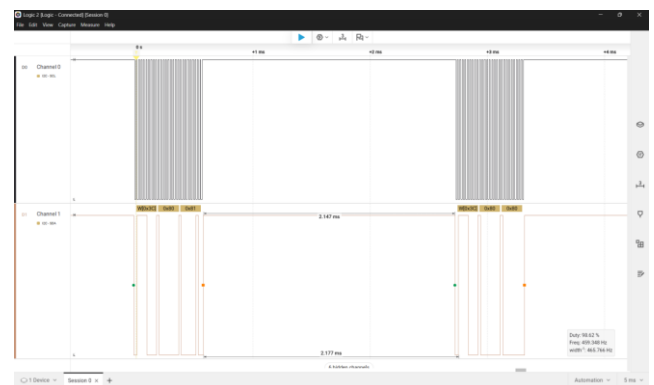


Figura 15. Opción 3, contraste: dos transacciones separadas por ≈ 2.15 ms.

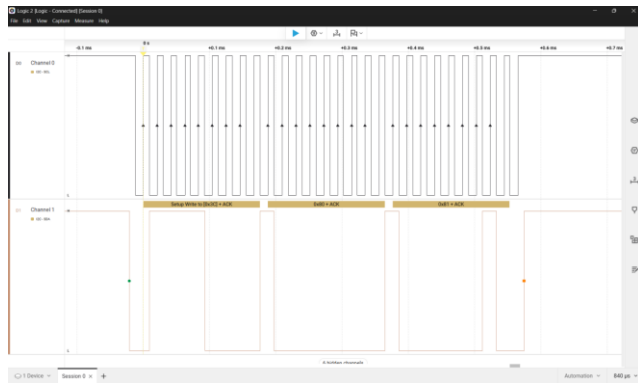


Figura 16. Primera transacción del contraste: 0x80 + 0x81.

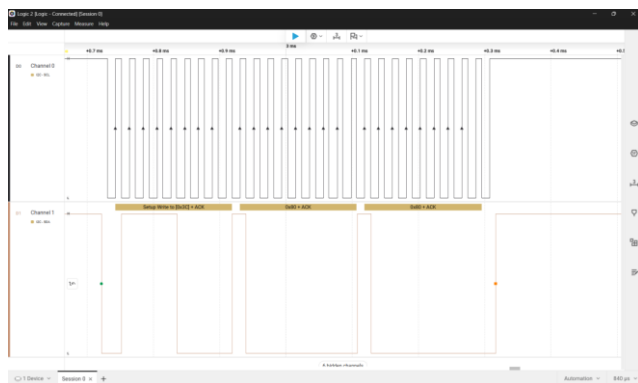


Figura 17. Segunda transacción del contraste: 0x80 + 0x80 (valor 128).

Invertir (opción 4). Con INVERT = 1 se envió 0x80 + 0xA7 (Figura 18) y la pantalla mostró el texto con fondo claro (Figura 19). Con INVERT = 0 se envió 0x80 + 0xA6 (Figura 20) y la pantalla regresó a la visualización normal (Figura 21).

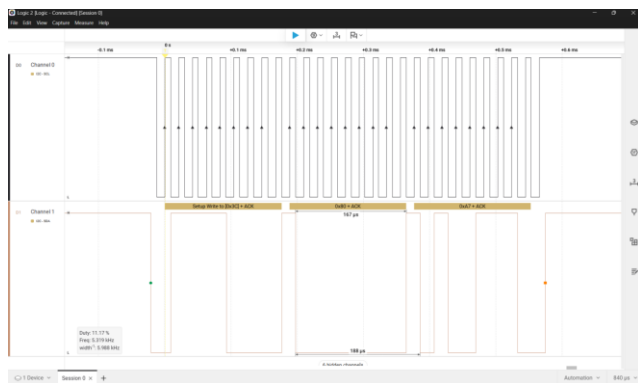


Figura 18. Invertir = 1: 0x80 + 0xA7 (Inverse Display).

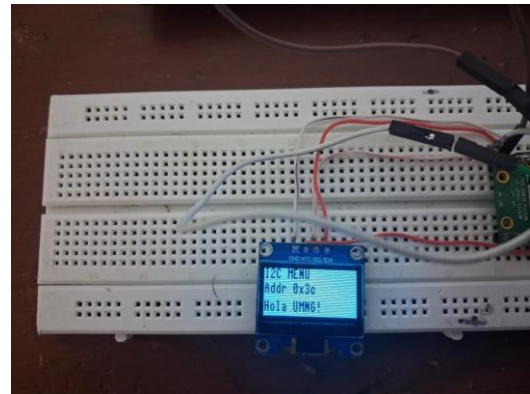


Figura 19. Pantalla en modo invertido.

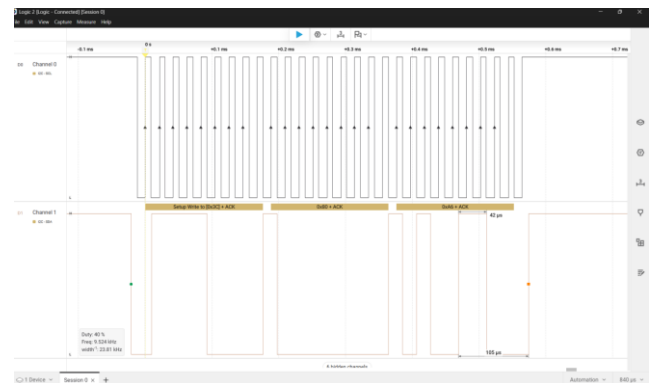


Figura 20. Invertir = 0: 0x80 + 0xA6 (Normal Display).

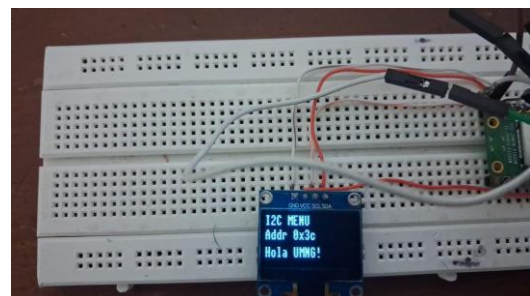


Figura 21. Pantalla en modo normal.

Texto/animación (opciones 6 y 7). La primera transacción contiene Write [0x3C], 0x80 y 0x03. La siguiente contiene Write [0x3C], el byte de control 0x40 y una ráfaga de datos (0x00, 0x00, 0x41, 0x7F, 0x7F, 0x41, ...), todos con ACK (Figura 22).

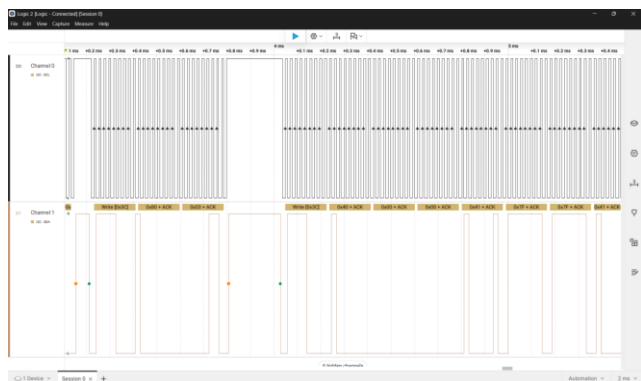


Figura 22. Texto/animación: byte de control 0x40 seguido de datos para la GDDRAM.

Tabla 3. Resumen de los códigos observados en el menú de la OLED.

Opción	Bytes observados tras la dirección	Función (hoja de datos [2])
Apagar	0x80, 0xAE	Set Display OFF
Encender	0x80, 0xAF	Set Display ON
Contraste (128)	0x80, 0x81 y luego 0x80, 0x80	Set Contrast Control (comando + valor)
Invertir = 1	0x80, 0xA7	Inverse Display
Invertir = 0	0x80, 0xA6	Normal Display
Texto/animación	0x80, 0x03 y luego 0x40, 0x00, 0x00, 0x41, 0x7F, ...	Comando de columna y escritura de datos en la GDDRAM

X. ANÁLISIS DE RESULTADOS

Estructura de la trama. En todas las capturas se reconoce la secuencia START, ocho bits de dirección con R/W, noveno bit de reconocimiento y STOP. El octeto 0x78 se obtiene desplazando la dirección 0x3C (0111100₂) una posición a la izquierda y agregando R/W = 0, es decir, 0111 1000₂. Para la dirección 0x3D el mismo procedimiento da 0x7A (0111 1010₂), valor que corresponde a lo decodificado como «Write to [0x3D]».

ACK frente a NACK. Entre las dos capturas se mantienen el START, el STOP, los nueve pulsos de reloj y la frecuencia de SCL (83.07 kHz frente a 83.43 kHz, Tabla 1). Cambian el octeto transmitido (0x78 frente a 0x7A) y el nivel de SDA en el noveno bit: 0 para el ACK y 1 para el NACK. La duración del noveno bit es prácticamente igual en ambos casos (13.004 μ s frente a 12.986 μ s, Tabla 2), por lo que la respuesta del esclavo no altera la temporización del bus.

Por qué una dirección incorrecta no recibe ACK. Durante los ocho primeros pulsos cada esclavo compara la dirección recibida con la suya. La OLED responde en 0x3C, y ese valor depende del pin SA0 del SSD1306, que solo admite 0x3C o 0x3D [2]. Con 0x3D, en este montaje, ningún dispositivo coincide y por tanto ninguno fuerza SDA a nivel bajo en el noveno pulso. El maestro libera la línea y las resistencias de pull-up la mantienen en alto, lo que se interpreta como NACK. En MicroPython esto se traduce en una excepción OSError, como se prevé en el código de prueba.

Frecuencia de SCL. Con una configuración de 100 kHz se midió un período de 12.038 μ s (83.07 kHz) y 11.987 μ s (83.43 kHz), aproximadamente 17 % por debajo del valor configurado. La duración del octeto completo, 84.103 μ s, equivale a unos 10.5 μ s por bit ($\approx$ 95 kHz), mientras que el noveno pulso dura cerca de 13 μ s. Esto indica que la medición depende del pulso elegido con los cursores y que el período no es perfectamente uniforme dentro de la trama. Una posible causa es que la frecuencia real del periférico PC se obtiene mediante divisores enteros del reloj del sistema y que los tiempos de subida de las líneas con pull-up modifican los flancos. En la parte del menú, configurada a 50 kHz, se midieron 21 μ s ($\approx$ 47.6 kHz), cerca de 5 % menos. Estos resultados muestran que la frecuencia configurada en el software es una referencia y que la frecuencia real debe verificarse con el analizador. La frecuencia de muestreo usada debe cumplir el criterio de $\geq 10 \times f_{SCL}$ para que estas mediciones sean confiables.

Escaneo. La traza de `i2c.scan()` muestra sondeos consecutivos (0x3B, 0x3C, 0x3D, 0x3E...) de la forma START + dirección + bit 9 + STOP. Solo el sondeo a 0x3C recibe ACK, lo que coincide con la lista ['0x3c'] de la consola y confirma que la OLED es el único dispositivo presente en el bus. Los demás sondeos terminan en NAK sin transmitir datos adicionales.

Comandos de la OLED. Todos los bytes fueron reconocidos con ACK, lo que indica que el SSD1306 aceptó cada transmisión. Cada comando va precedido por el byte de control 0x80 (Co = 1, D/C# = 0) y los datos de pantalla por 0x40 (Co = 0, D/C# = 1) [2]. Los códigos observados (0xAE, 0xAF, 0x81, 0xA7 y 0xA6) coinciden con los de la Tabla 3 y con el efecto visual de las fotografías: pantalla negra, pantalla con texto, texto sobre fondo claro y regreso a la visualización normal. El contraste es un comando de dos bytes: 0x81 indica la función y el segundo byte, 0x80, fija el valor 128. En este programa cada byte de comando se envía con su propio byte de control 0x80, mientras que para el texto un solo 0x40 va seguido de una ráfaga de datos para la GDDRAM. Por eso las transferencias de datos son más eficientes que los comandos, que repiten el byte de control. El intervalo de unos 2.15 ms entre las dos transacciones del contraste es atribuible al tiempo de ejecución del intérprete MicroPython entre llamadas sucesivas. Finalmente, según la hoja de datos, los comandos entre 0x00 y 0x0F, como el 0x03 observado antes de los datos, ajustan la columna inicial en modo de direccionamiento por página [2].

XI. ANEXOS

Enlace GitHub:
<https://github.com/Salomeblanc/Reconocimiento-de-tramas-I2C-con-analizador-logico>

XII. CONCLUSIONES

Se identificó experimentalmente la estructura de una transacción PC de escritura: START, octeto de dirección con R/W, bit de reconocimiento y STOP. Para la OLED en 0x3C el octeto observado fue 0x78, resultado de desplazar la dirección una posición y agregar R/W = 0.

La comparación entre las capturas con 0x3C y 0x3D mostró que solo cambian el octeto (0x78 y 0x7A) y el nivel de SDA en el noveno bit (ACK = 0 y NACK = 1). La OLED únicamente reconoce su propia dirección, por lo que ningún otro dispositivo llevó SDA a nivel bajo con 0x3D.

La frecuencia medida de SCL fue de 83.07 kHz para 100 kHz configurados y de unos 47.6 kHz para 50 kHz configurados. La frecuencia real del bus puede diferir de la programada, por lo que conviene medirla con el analizador lógico.

El método `i2c.scan()` detectó únicamente la dirección 0x3C, y la traza lo confirmó con un ACK entre sondeos con NAK. Esto valida el escaneo como herramienta para comprobar qué dispositivos están presentes en el bus.

Los bytes observados para apagar, encender, contraste, inversión y escritura de texto coinciden con la hoja de datos del SSD1306 y con el efecto visible en la pantalla. Se distinguieron el byte de control 0x80 para comandos y 0x40 para datos. El analizador lógico resultó útil para relacionar el código del programa con las señales del bus.

XIII. REFERENCIAS

- [1] NXP Semiconductors, "UM10204: I²C-bus specification and user manual," Rev. 7.0, Oct. 2021.
- [2] Solomon Systech, "SSD1306: 128 × 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller," Datasheet, Rev. 1.1, Apr. 2008.
- [3] Raspberry Pi Ltd, "Raspberry Pi Pico 2 W," Datasheet. [En línea]. Disponible:
<https://www.raspberrypi.com/documentation/microcontrollers/>
- [4] MicroPython, "class I2C – a two-wire serial protocol," Documentación de la biblioteca machine. [En línea]. Disponible:
<https://docs.micropython.org/en/latest/library/machine.I2C.html>
- [5] Saleae, "Logic 2 Software," Saleae Inc. [En línea]. Disponible:
<https://www.saleae.com/>
- [6] J. Rugeles, "Reconocimiento de tramas I2C con analizador lógico," guía de laboratorio de Comunicaciones Digitales, Universidad Militar Nueva Granada. Código: <https://github.com/jrugeles/I2C>